Pre-Lab 05 Answers

Data Visualization · CSV inspection and D3 interaction

Section 4: Inspecting numeric CSV fields

The page shows 5 raw rows and 4 valid ones. The row it drops is `Overflow,not-a-number,40,15`.

In the first `console.table(rows)`, every value is a string. `d3.csv` doesn't guess types, so "Reading Hall" comes through as `East: "22"`, `North: "34"`, `Visitors: "16"`. They look like numbers, but `"22" + 1` gives you `"221"`. The second table is different. The rows have new names (`name, x, y, count`), the values are real numbers, and Overflow isn't there.

The code does this in two steps:

1. `.map()` runs `Number()` on `East`, `North` and `Visitors`. `Number()` doesn't throw when it gets junk. It returns `NaN`, so Overflow ends up with `x: NaN`.
2. `.filter()` keeps a row only if all three values pass `Number.isFinite` (which `NaN` fails), the count is 0 or more, and x and y are both between 0 and 100. Empty Lab has 0 visitors and still passes. Zero visitors is a real reading, so it shouldn't be dropped.

There's a catch the practice file doesn't hit: `Number("")` is `0`, not `NaN`. That means a blank cell would pass the check as a zero. If the real data has blanks, test for them first with something like `row.East.trim() !== ""`.

For Lab 05, the Johns Hopkins file (`time_series_covid19_confirmed_global.csv`) is shaped differently. It starts with four metadata columns, `Province/State, Country/Region, Lat, Long`, and then has one column per day. The day columns are named in US `M/D/YY` format with no leading zeros, like `1/22/20` and `4/5/20`. Our lab date is written the Vietnamese way. `05/04/2020` means 5 April 2020, so the column we want is `4/5/20`. If you read the date the US way, you'd pick `5/4/20` instead. That column also exists, so nothing errors and you'd just plot 4 May without noticing.

```
const [d, m, y] = '05/04/2020'.split('/');
const key = `${+m}/${+d}/${y.slice(2)}`;   // "4/5/20"
console.log(rows.columns.includes(key));    // should be true
```

After that, the same pattern works. Convert `Lat`, `Long` and `row[key]` with `Number()`, filter, and compare the raw and valid counts before drawing anything. A few rows in this file aren't tied to a single place, so check whether their `Lat`/`Long` are blank. If they are, they'd end up at (0, 0) off the coast of Africa.

Answer 1: How are `event` and `d` different in a D3 event handler?

D3 v7 calls a handler like `showTooltip(event, d)` with two separate things:

- `event` is the browser's event object. It describes what the user did and where: `event.clientX`/`clientY` give the pointer position, and `event.currentTarget` is the circle element that got the event. `moveTooltip(event)` needs only this to place the tooltip, and `hideTooltip` uses `currentTarget` to reset the outline.

- `d` is the data row attached to that circle by `.data(spaces, d => d.id)`, e.g. `{id: 'R4', name: 'Engineering Hub', visitors: 44, …}`. `content(d)` uses only this to fill the tooltip text.

So `event` tells you about the interaction and the element, and `d` tells you which record it is. The order matters too. In D3 v5 the handler got `(d, i)` and the event was in the global `d3.event`. Copying an old tutorial gets these mixed up.

Answer 2: Why do we need to hide or reposition a tooltip?

The tooltip is a single shared `div` with `position: fixed`, and it's hidden (`display: none`) until a circle is hovered. It needs both behaviours:

- Hide it on `pointerleave`. If you don't, the last room's numbers stay on screen after the pointer moves away. Someone could read them as belonging to whatever circle is underneath, and the box also covers other marks.
- Move it on `pointermove`. It should follow the pointer so it's clearly about the circle you're on. The code also keeps it on screen: `Math.min(clientX + 15, innerWidth - 240)` stops it running off the right edge, and `Math.max(10, clientY - 50)` stops it going above the top.

The CSS has `pointer-events: none` on the tooltip for a related reason. Without it, the tooltip can end up under the cursor and take the pointer from the circle. The circle then fires `pointerleave`, the tooltip hides, the circle fires `pointerenter` again, and it flickers.

Answer 3: What evidence proves every valid data row has a visual mark?

The worked example's section E does it with a count check in the console:

```
console.log('Expected points:', spaces.length);                      // 5
console.log('Actual points:', plot.selectAll('circle.spot').size());  // 5
```

If both lines print 5, there's one circle per row. A matching count isn't quite enough on its own, so I'd also check:

- No circle has `cx`, `cy` or `r` set to `NaN`. Those circles are in the DOM but never draw. Searching for `NaN` in the Elements panel will catch them.
- Hovering each circle shows the right row in the tooltip. That shows the data actually got joined to the circles, which the count doesn't prove.
- For the CSV version, compare `valid.length` with the number of circles, not `rows.length`. The rejected rows aren't supposed to get a mark.

Pre-lab boundary

The worked example uses made-up study rooms to show how the interaction works. In Lab 05 my partner and I have to choose how to show the real COVID data ourselves, then test that choice and explain it. We can reuse the techniques, but the design has to be ours.